

Tema 5: Estructuras Repetitivas y Bucles Controlados

MOMENTO 1: PRÁCTICA

LA AUTOMATIZACIÓN DE PROCESOS REPETITIVOS EN EL ENTORNO REAL

Imagine que al finalizar la jornada académica en el Centro de Educación Alternativa "Herman Ortiz Camargo", se le encomienda al encargado de sistemas registrar la asistencia de una lista física de 45 estudiantes en la plataforma digital. El proceso manual requiere que el operador lea el primer nombre, verifique su estado, marque la casilla correspondiente en la pantalla y pase al siguiente registro. Este procedimiento se repite de manera idéntica y sistemática para el estudiante 2, el estudiante 3, el estudiante 4, y así sucesivamente hasta llegar al número 45.

En este escenario real, se identifican con total claridad tres factores fundamentales que definen cualquier tarea iterativa:

1. Un **punto de inicio definido** (comenzar estrictamente con el estudiante número 1).
2. Un **mecanismo de avance constante** (pasar de un estudiante al siguiente sumando una unidad a la posición actual).
3. Una **condición de parada absoluta** (detener el proceso inmediatamente después de procesar al estudiante número 45).

Si tuviéramos que escribir un programa en la computadora para realizar esta tarea sin conocer las estructuras repetitivas, nos veríamos obligados a copiar y pegar las mismas líneas de código 45 veces consecutivas. Si la lista aumentara a 1,000 estudiantes, el archivo de código fuente se volvería inmanejable, ineficiente y propenso a errores críticos de digitación. Aquí es donde radica el verdadero poder de la informática: delegar a la Unidad Central de Procesamiento (CPU) la ejecución veloz de tareas rutinarias mediante el uso de **bucles o ciclos condicionales**.

Analogía de Hardware: El Bucle Infinito del Microcontrolador

En el desarrollo de sistemas embebidos (como el sistema biométrico de asistencia con el módulo ESP32 que analizamos en la unidad anterior), el microcontrolador depende intrínsecamente de una estructura

repetitiva infinita conocida como ***void loop()***. Una vez encendido el dispositivo, este bucle ejecuta de forma permanente e ininterrumpida la lectura del sensor de huellas dactilares, esperando un pulso físico sin detenerse jamás, a menos que se corte el suministro eléctrico.

MOMENTO 2: TEORÍA

FUNDAMENTOS GENERALES, VARIABLES DE CONTROL Y TIPOS DE BUCLES

En el ámbito del diseño de algoritmos, las **estructuras repetitivas** (también llamadas lazos, bucles o ciclos) permiten ejecutar un bloque de instrucciones múltiples veces consecutivas basándose en el valor de verdad de una condición lógica determinada.

1. Elementos Esenciales de un Bucle

Para evitar que un ciclo se ejecute de forma descontrolada y sature la memoria del sistema, el flujo algorítmico debe incorporar y administrar con precisión matemática dos tipos de variables especiales:

- **El Contador:** Es una variable de tipo entero que incrementa o decrementa su valor en una cantidad constante en cada vuelta del ciclo. Su estructura matemática clásica se expresa como:

contador = contador + 1 o su operador compacto ***contador++***

- **El Acumulador o Sumador:** Es una variable numérica que incrementa su valor de forma variable, sumando o acumulando los montos obtenidos en cada iteración del bucle. Es fundamental para obtener subtotales o sumatorias totales. Su estructura matemática se denota como:

acumulador = acumulador + variableSuma o de forma compacta ***acumulador += variableSuma***

2. Clasificación Sintáctica de las Estructuras Repetitivas

Dependiendo del momento exacto en que se evalúe la condición lógica y de si se conoce o no de antemano el número exacto de vueltas, los lenguajes de programación modernos clasifican los bucles en tres grandes familias estructuradas:

Estructura Básica	Evaluación y Mecánica	Cuándo Utilizarla	Ejemplo en Pseudocódigo
MIENTRAS (While)	Pre-test: Evalúa la condición lógica al inicio. Si la condición es falsa desde el principio, el bloque interno nunca se ejecuta.	Cuando no se conoce el número exacto de iteraciones y el ciclo depende de un factor externo variable.	<pre> Mientras (sensor == 0) LeerSensor() FinMientras </pre>
HACER MIENTRAS (Do - While)	Post-test: Ejecuta el bloque interno primero y evalúa la condición al final. Garantiza que el código se ejecute al menos una vez obligatoriamente.	Ideal para validaciones de datos de entrada y despliegue de menús interactivos de usuario.	<pre> Hacer MostrarMenu() Mientras (opcion != 0) </pre>
PARA (For)	Ciclo Controlado: Agrupa en una sola línea la inicialización, la condición de parada y el incremento de la variable contador.	Cuando se conoce con total exactitud matemática el número de vueltas antes de ingresar al bucle.	<pre> Para i = 1 Hasta 45 ConPaso 1 RegistrarAsistencia(i) FinPara </pre>

REGLA DE ORO DE DESARROLLO: El Peligro del Bucle Infinito (Infinite Loop)

Un bucle infinito ocurre cuando la condición de control de un ciclo **Mientras** o **Hacer-Mientras** se mantiene permanentemente en estado Verdadero (**True**). Esto sucede casi siempre porque el desarrollador olvidó escribir la línea de incremento del contador (por ejemplo, omitió colocar **i++**) dentro del bloque de instrucciones. Como consecuencia, el procesador se queda atrapado eternamente ejecutando la misma sección de código, elevando el uso de la CPU al 100% y congelando la aplicación o el servidor web de forma inmediata.

3. Análisis Sintáctico en Código de Producción

Para comprender el comportamiento técnico de cada estructura, analizaremos el mismo problema resuelto con los dos enfoques más utilizados en el desarrollo de software actual: la estructura condicional indeterminada (**while**) y la estructura determinada (**for**). **A. Implementación Estructurada con Ciclo Mientras (While):**

```
entero contadorEstudiantes = 1; // Inicialización de la variable de control

mientras (contadorEstudiantes <= 5) {
    Imprimir("Procesando matrícula del estudiante número: " +
contadorEstudiantes);

    // LÍNEA CRÍTICA: Incremento constante para evitar bucle infinito
    contadorEstudiantes = contadorEstudiantes + 1;
}
Imprimir("Proceso de lote finalizado con éxito.");
```

B. Implementación Estructurada con Ciclo Para (For):

La estructura **for** compacta el diseño del algoritmo de forma elegante, reduciendo el riesgo de omisión de la línea de actualización del contador.

```
// Sintaxis normalizada: (Inicialización; Condición de control; Incremento
diario)
para (entero i = 1; i <= 5; i++) {
    Imprimir("Generando credencial técnica ID: " + i);
}
Imprimir("Impresión automatizada completada.");
```

MOMENTO 3: VALORACIÓN

EFICIENCIA ALGORÍTMICA Y OPTIMIZACIÓN DE RECURSOS DE PROCESAMIENTO

En la práctica profesional de las ciencias de la computación, el uso correcto de las estructuras repetitivas es el pilar que separa un software de nivel aficionado de una solución escalable de nivel empresarial. Un procesador puede realizar miles de millones de cálculos sencillos por segundo, pero la mala planificación de un bucle puede

degradar catastróficamente la experiencia de usuario (UX) o elevar drásticamente los costos de alojamiento en la nube.

Pensemos detenidamente en la arquitectura de un sistema de gestión escolar alojado en plataformas en la nube como Render o AWS. Si el código encargado de buscar reportes de calificaciones ejecuta consultas directas a la base de datos PostgreSQL *dentro* de un bucle para de 500 iteraciones (una práctica ineficiente conocida técnicamente como el problema de consulta N+1), el servidor saturará sus canales de conexión en cuestión de segundos. Esto causaría retrasos severos en la carga y la posterior caída del servicio del backend.

Asimismo, en el contexto de la seguridad de la información y la auditoría de sistemas, comprender los ciclos es vital. Los ataques de denegación de servicio (DoS) o los algoritmos de fuerza bruta utilizados por crackers para descifrar contraseñas no son más que bucles estructurados a altas velocidades diseñados para probar millones de combinaciones consecutivas hasta forzar una condición verdadera. Como diseñadores de software, nuestra obligación moral y técnica es estructurar bloques iterativos limpios, acotados y con condiciones de salida robustas que garanticen la estabilidad de la infraestructura digital.

MOMENTO 4: PRODUCCIÓN

CUADERNO DE TRABAJO: LABORATORIO DE ITERACIONES CONTROLADAS

Complete de forma manuscrita los siguientes ejercicios técnicos de abstracción y lógica en su cuaderno de avance para validar sus competencias en el diseño de bucles estructurados.

Actividad 1: Desarrollo Práctico de Pseudocódigo (Control de Inventario)

Un almacén técnico de computadoras en la zona comercial requiere un script automatizado para su terminal de comandos. El programa debe pedirle al usuario que ingrese de forma secuencial el precio de costo de exactamente **6 componentes de hardware** (memorias RAM, discos sólidos, etc.). El sistema debe calcular automáticamente de manera interactiva dos resultados finales utilizando variables de control: El costo total acumulado de la compra y el promedio exacto del precio de los componentes.

Requisito obligatorio: Desarrolle el algoritmo utilizando exclusivamente la estructura repetitiva **Mientras** o **Para**.

Actividad 2: Prueba de Escritorio Analítica (Seguimiento Mental de Variables)

Analice minuciosamente el comportamiento lógico del siguiente algoritmo simulando el hilo de ejecución de la CPU. Complete la tabla de variables adjunta con los cambios de valores en cada iteración:

Variables de ejecución inicial:

```
entero contadorCiclos = 1
entero acumuladorSuma = 0
entero limiteControl = 4
```

Algoritmo Estructurado:

```
Mientras (contadorCiclos < limiteControl) Hacer
    acumuladorSuma = acumuladorSuma + (contadorCiclos * 2)
    contadorCiclos = contadorCiclos + 1
FinMientras

Imprimir("El resultado final del acumulador es: " + acumuladorSuma)
```

Número de Vuelta	Valor de contadorCiclos	Evaluación Lógica (¿< limiteControl?)	Valor de acumuladorSuma
Inicio	1	(No aplica)	0
Vuelta 1	_____	¿ 1 < 4 ? → Verdadero	_____
Vuelta 2	_____	_____	_____
Vuelta 3	_____	_____	_____
Final del Ciclo	¿Sigue cumpliendo?	_____	VALOR FINAL: _____

Actividad 3: Auditoría de Sistemas e Identificación de Errores Lógicos (Debugging)

Un estudiante en prácticas del laboratorio técnico diseñó un módulo interactivo para validar las contraseñas de seguridad de los usuarios del sistema. Sin embargo, al realizar las pruebas de calidad, el programa se cuelga inmediatamente ingresando en un bucle infinito que congela la computadora. Analice el código fuente e identifique la falla:

```
cadena claveCorrecta = "Cea2026"
cadena claveIngresada = ""
entero intentosRealizados = 0

// El código pretende dar oportunidades hasta adivinar la clave
mientras (claveIngresada != claveCorrecta) {
    Imprimir("Por favor introduzca la contraseña del sistema:");
    claveIngresada = "Cea2026";

    // El estudiante quería que el programa termine si pasa los 3 intentos,
    pero algo falló.
    if (intentosRealizados == 3) {
        Imprimir("Acceso bloqueado por seguridad institucional.");
    }
}
```

Responda de forma detallada:

1. Explique técnicamente por qué la validación de la variable **intentosRealizados** jamás se cumplirá dentro de la estructura condicional.
2. Reescriba el fragmento de código corrigiendo la lógica e incorporando el operador de incremento adecuado para limitar estrictamente los intentos a un máximo de 3 vueltas reales.

Actividad 4: Implementación Gráfica Automatizada (Flowgorithm Challenge)

Abra el entorno de desarrollo *Flowgorithm* en los terminales del laboratorio de computación del CEA. Construya un diagrama de flujo interactivo y profesional que cumpla rigurosamente con los siguientes requerimientos de software:

- El sistema debe solicitarle al usuario que introduzca por teclado un número entero positivo correspondiente a una tabla de multiplicar (por ejemplo, si introduce el 7, el sistema generará la tabla del 7).
- Utilizando un bucle controlado de tipo **Para** (For), el programa debe calcular de manera automatizada e imprimir en pantalla los resultados del producto desde el factor 1 hasta el factor 12 de forma ordenada y limpia (ejemplo visual en pantalla: "7 x 1 = 7", "7 x 2 = 14", etc.).
- Asegúrese de inicializar correctamente las variables de control y documentar el flujo con mensajes descriptivos orientados al usuario final, garantizando los estándares de calidad de interfaces estipulados en la academia.